Car Rental System

Hussein Mohamud, Aidan Elkins, Emily Bailey, William Varda, Rami Ibraheem

System Description:

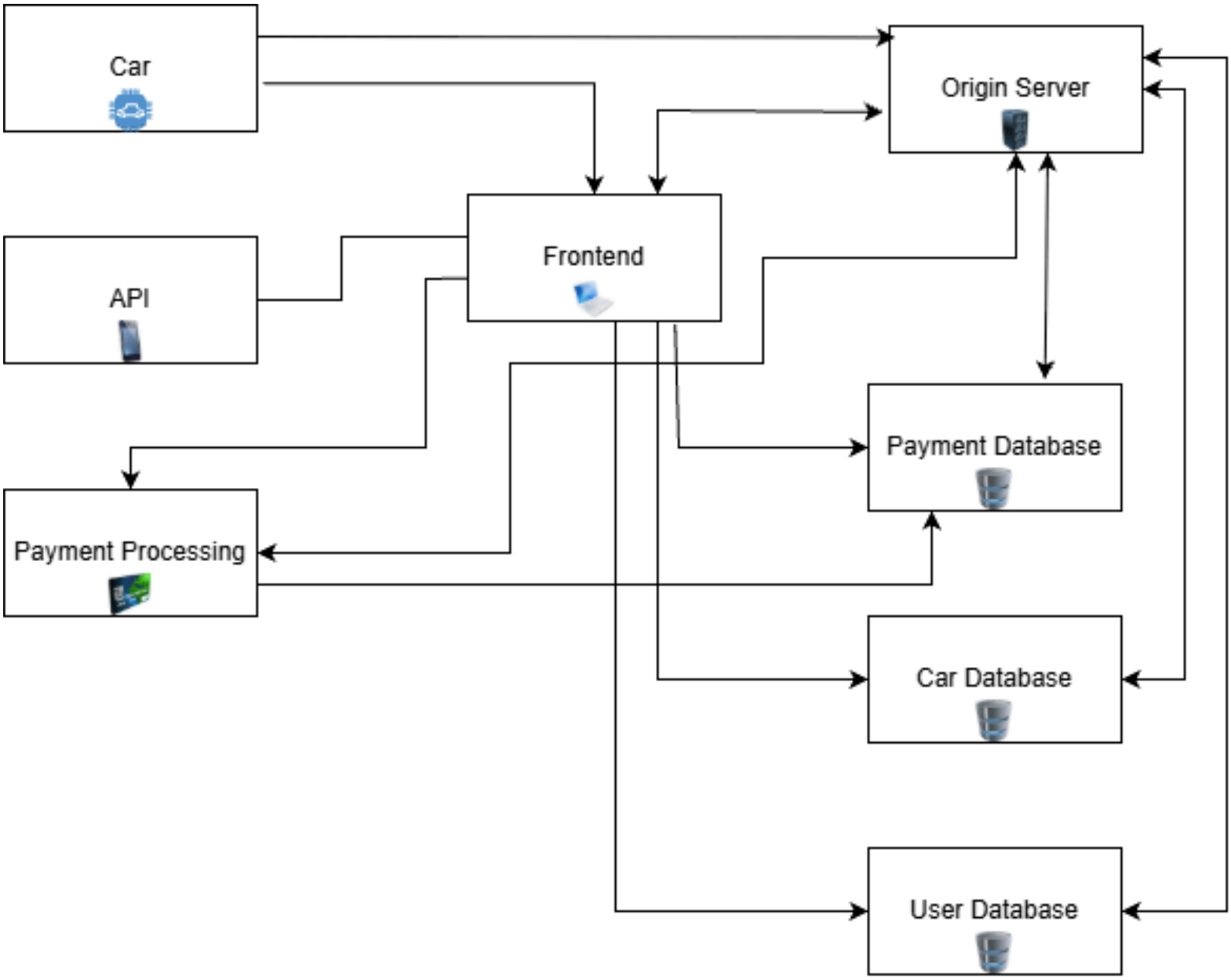
This system handles car rentals and overall day-to-day management for BeAvis. It has the option of being accessible via both website and mobile application (with latest IOS or Android version), each with the same utilities. Users can enable GPS to view nearby locations as well as their direction once a rental is requested.

Users must create an account with their email and a password to make rentals. After verifying an email, their account will be created and they will then have the option to enable two-factor authentication with a separate authenticator app at any time, although it is not mandatory. Users can make rentals, view rental history, find locations, and link their method of payment. Payment information is secure and in a different location from other data in case of breach.

Rental agreement contracts are also given and signed here; the contracts will be kept secure along with customer signatures.

Employees can log into the system and are able to access customers' rental status and contracts, see cars available for rental, and update car status.

Software Architecture Overview:



SWA Description:

Component name: **Car**
Purpose: Processes requests for Car information, communicating with Car database
Connects to: Frontend, Origin Server

Component name: **API**
Purpose: Calls external APIs for routing requests to back end servers
Connects to: Frontend

Component name: **Payment Processing**
Purpose: Processes payments made, as well as calls external payment systems
Connects to: Payment Database

Component name: **Frontend**
Purpose: User interface, client side requests for information and updates
Connects to: Origin Server, Payment Database, Car Database, Customer Database, Payment Processing, API

Component name: **Origin Server**
Purpose: Coordinates cross database communication, store and update information
Connects to: Frontend, Payment Database, Car Database, Customer Database, Payment Processing

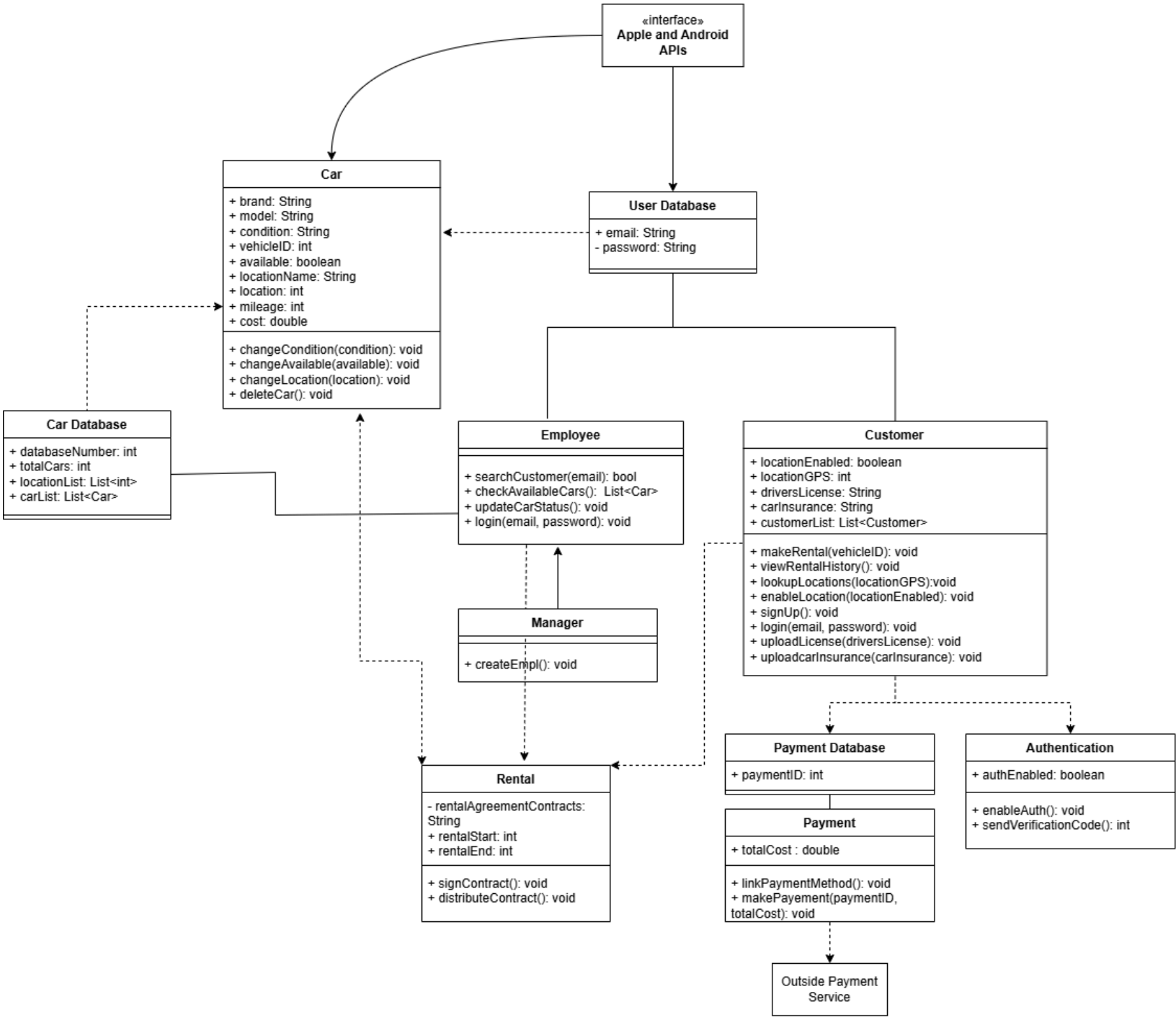
---

Component name: **Payment Database**
Purpose: Database isolation protects user information and stores payment data
Connects to: Frontend, Origin Server, Payment Processing

Component name: **Car Database**
Purpose: Stores total car information
Connects to: Frontend, Origin Server

Component name: **Customer Database**
Purpose: Database isolation protects user information and store user data
Connects to: Frontend, Origin Server

UML Diagram


«interface»
Apple and Android APIs
Car
+ brand: String
+ model: String
+ condition: String
+ vehicleID: int
+ available: boolean
+ locationName: String
+ location: int
+ mileage: int
+ cost: double
+ changeCondition(condition): void
+ changeAvailable(available): void
+ changeLocation(location): void
+ deleteCar(): void
User Database
+ email: String
- password: String
Car Database
+ databaseNumber: int
+ totalCars: int
+ locationList: List<int>
+ carList: List<Car>
Employee
+ searchCustomer(email): bool
+ checkAvailableCars(): List<Car>
+ updateCarStatus(): void
+ login(email, password): void
Customer
+ locationEnabled: boolean
+ locationGPS: int
+ driversLicense: String
+ carInsurance: String
+ customerList: List<Customer>
+ makeRental(vehicleID): void
+ viewRentalHistory(): void
+ lookupLocations(locationGPS):void
+ enableLocation(locationEnabled): void
+ signUp(): void
+ login(email, password): void
+ uploadLicense(driversLicense): void
+ uploadcarInsurance(carInsurance): void
Manager
+ createEmpl(): void
Rental
- rentalAgreementContracts: String
+ rentalStart: int
+ rentalEnd: int
+ signContract(): void
+ distributeContract(): void
Payment Database
+ paymentID: int
Payment
+ totalCost : double
+ linkPaymentMethod(): void
+ makePayement(paymentID, totalCost): void
Authentication
+ authEnabled: boolean
+ enableAuth(): void
+ sendVerificationCode(): int
Outside Payment Service

Description of Classes, Associated Attributes & Operations

Class name: **Car Database**
Purpose: Abstract class to store and update car object within lists
Parameters
+ totalCars: int
+ locationList: List<int>
+ carList: List<Car>

No Methods

Class name: **Car**
Purpose: To create and update all stored car objects
Parameters
+ brand: String
+ model: String
+ condition: String
+ vehicleID: int
+ available: boolean
+ locationName: String
+ location: int
+ mileage: int
+ cost: double

Methods
+ changeCondition(condition): void
    updates condition for vehicle
+ changeAvailable(available): void
    updates availability for vehicle
+ changeLocation(location): void
    updates location for vehicle
+ deleteCar(): void
    removes car from list

Class name: **User Database**
Purpose: Abstract class for both user types
Parameters

+ email: String
- password: String

No Methods

Class name: **Employee**
Purpose: Inherits from, contains employee objects and associated roles
No Parameters

Methods
+ searchCustomer(email): bool
    finds customer in within list, returns true if found
+ checkAvailableCars(): List<Car>
    searches list for cars with list, returns list of available
+ updateCarStatus(): void
    updates car availability status
+ login(email, password): void
    logs employees in to access workplace methods

Class name: **Manager**
Purpose: Inherits from Employee
No Parameters

Methods
+ createEmpl(): void
    creates new employee object with sign in capabilities

Class name: **Customer**
Purpose: Inherited from User, abstract class to create and update all stored customer objects
Parameters
+ locationEnabled: boolean
+ locationGPS: int
+ driversLicense: String
+ carInsurance: String
+ customerList: List<Customer>

Methods
+ makeRental(vehicleID): void
    start rental process with vin
+ viewRentalHistory(): void

connects to user's prior rental history
+ lookupLocations(locationGPS): void
searches car list to find cars based on location
+ enableLocation(locationEnabled): void
alters location boolean true for on false for off
+ signUp(): void
initiates creation of new customer object - new account creation with input checks to verify valid email and password (calls on sendVerificationCode() method in Authentication class)
+ login(email, password): void
searches customer database for current customer login info
+ uploadLicense(driversLicense): void
uploads string to verify rental requirement for drivers license
+ uploadcarInsurance(carInsurance): void
uploads string to verify rental requirement for car insurance

---

Class name: **Rental**
Purpose: Tracks active rental contracts (private to maintain secure information) to complete signing, processing, and distributing
Parameters
- rentalAgreementContracts: String
+ rentalStart: int
+ rentalEnd: int

Methods
+ signContract(): void
completes virtual e-signature request
+ distributeContract(): void
signed contract distributed

Class name: **Payment Database**
Purpose: Abstract class for payment objects
Parameters
+ paymentID: int

No Methods

Class name: **Payment**
Purpose: Secured by separate class and private methods, class functions to process payments and handle payment information

Parameters
+ totalCost: double

Methods
- linkPaymentMethod(): void
    allows user to link their payment method to their account for purchases
- makePayment(): void
    allows for users to make the payment. Also contacts external payment service

---

Class name: **Authentication**
Purpose: Allows two factor authentication optional service and runs email authentication for new account creation
Parameters
+ authEnabled: boolean

Methods
+ enableAuth(): void
    allows for the user to enable 2 factor authentication
+ sendVerificationCode(): int
    sends the user's email a verification code

Verification Test Plan

TEST EXPLANATION:

In these tests, we are testing the changeLocation function of the Car class. In the Unit test we do this by making a Car object, named car, and attempt to change the location of the car by calling the respective function. If the location returns “San_Diego_Mile_of_Cars”, it is considered passing, otherwise it is considered a failure.

In the Integration test, we are testing the same changeLocation function along with the relationship between the Employee Class and the Car Database that allows employees access to complete alterations.

In the Systems Test we complete a broad plaintext overview of the interaction where changeLocation would be used in a real world scenario and possible results.

Test	Input Category	Test Input	Expected Output
1	Valid Store Location	San_Diego_Mile_of_Cars	Confirms account association
2	Invalid Store Location	Racoon_City_Storage	Displays “Invalid Location”
3	Typos Store Location	San_Diego_Mill_of_Carp	Displays “Invalid Location”
4	Empty Spaces	“ “	Displays “Invalid Location”

Unit Test ChangeLocation(location)

```
BEGIN
SET
Car car = civic;
String location = “San_Diego_Mile_of_Cars”

car.changeLocation(location);

If (car.location == “San_Diego”)
        return PASS
Else
        return FAIL

END
```

Integration Test ChangeLocation(location)
BEGIN
SET
Employee empl;
Car car = civic;
car.location = “Hemet”;

empl.login(“johndoe@company.com”, “P@s$w0rd”);

String newLocation = “San_Diego”

car.changeLocation(newLocation);

If (car.location == “San_Diego”)
    return PASS
Else
    return FAIL

END

Systems Test
An employee logs into the software using email and password. They wish to reflect a customer returning a rented Civic from one location and dropped off at an alternate location. They select a Civic currently in the company catalog and choose to change its location from Hemet to San Diego.

TEST EXPLANATION:

In these tests, we are testing the searchCustomer function of the Employee class. In the Unit test we do this by making an Employee object, named empl, and attempt to search a customer based on email associated with existing accounts. If the email is found it returns true, this is considered passing, otherwise it returns false, this is considered a failure.

In the Integration test, we are testing the same searchCustomer function along with the relationship between the Employee Class and the Customer Database that allows employees access to customer profiles.

In the Systems Test we complete a broad plaintext overview of the interaction where searchCustomer would be used in a real world scenario and possible results.

Test	Input Category	Test Input	Expected Output
1	Valid Customer Email	janedoe@gmail.com	Confirms account association
2	Invalid Customer email	actionman@jwick.org	Displays “No account found”
3	Typos Email Search	jannedough@gmail.com	Displays “No account found”
4	Empty Spaces	“ “	Displays “No account found”
5	Exceeds Character Limit	[254+characters]@gmail.com	Displays “Exceeds character limit”

Unit Test searchCustomer(email)

```
BEGIN
SET
Employee empl;
String custEmail = “janedoe@gmail.com”;

If(custEmail.length > 264)
        return FAIL

bool found = empl.searchCustomer(custEmail);

If (found == true)
        return PASS;
Else
        return FAIL;

END
```

Integration Test searchCustomer(email)

```
BEGIN
SET
Employee empl;
empl.login(“johndoe@company.com”, “P@s$w0rd”);

Customer cust = newCust();
cust.email = “janedoe@gmail.com”;
```

```
bool found = empl.searchCustomer(cust.email);

If (found == true)
        return PASS;
Else
        return FAIL;

END
```

Systems Test

A customer approaches the car rental employee wanting to verify that their account is in the system. The employee logs into the system, asks for the customer's email associated with the account: janedoe@gmail.com. The employee enters the email and performs a search of the full customer database to confirm the account. Result shows a true for found account or a fail if the email is not connected to any existing account within the database.

Databases

Car_	
Vehicle ID (PK)	INTEGER
Brand	VARCHAR(50)
Model	VARCHAR(50)
Location ID	INTEGER
Availability	BOOLEAN

Example(s): (true = available)

1738 - Honda - Civic - 32117 - true

User_	
Email (PK)	VARCHAR(254)
Vehicle ID (FK)	INTEGER
Drivers License	VARCHAR(50)
Location ID	INTEGER

Example(s):

For a customer with location enabled & active rental contract:

janedoe@gmail.com - 1738 - D12345678 - 32117

For a customer without location enabled & no active rental contract:

janedoe@gmail.com - NULL - D12345678 - NULL

For employee:

johnphilip@rentAcar.com - NULL - P1234567 - 32117

Payment_	
PaymentID (PK)	INTEGER
Email (FK)	VARCHAR(254)
TotalCost	DECIMAL(10,2)

Example(s): 123456 - janedoe@gmail.com - $325.50

SQL Query Examples :

1) **Adding new Cars to Car_ Database:**

   INSERT INTO Car_ (Vehicle ID, Brand, Model, Location_ID)

   VALUES

   (1738, Honda, Civic, 32117)

   (8675, Toyota, Rav4, 32117)

   (2017, Toyota, Corolla, 16500)

2) **Finding cars by location id and availability:**

   CREATE VIEW by_Location AS

   SELECT [Location ID]

   FROM Car_

   WHERE [Location ID] = '32117'

   CREATE VIEW by_Availability AS

   SELECT [Availability]

   FROM Car_

   WHERE [Availability] = 'true'

3) **Removing cars from rentable inventory (done on individual basis by vehicle ID):**

   DELETE FROM Car_

   WHERE [Vehicle ID] = '1738'

4) **Consolidating 2 rental locations into 1, all cars from one location moved to another:**

   UPDATE Car_

   SET [Location ID] = '32117'

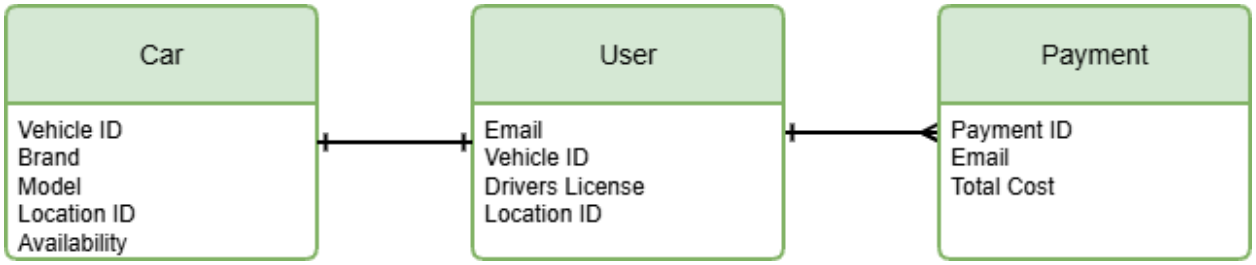
   WHERE [Location ID] = '16500'

Data Strategy & Justification

We chose to go with using SQL as our data management strategy. We decided that it would be easier to format and read our data, as well as it had better searching properties for our specific databases while maintaining relationships between normalized data. These benefits are enabled by primary and foreign keys that help us navigate in the databases and ensures ease of access to necessary values. We chose to include three databases in our design, those being:

1. Payment Database
2. Car Database
3. User Database

Entity Relationship Diagram



In the Payment Database, we store the paymentID for each transaction to enable backtracking/history viewing, a key to that user's email, and the cost of each transaction.

In the Car Database we store vehicleID, brand, model, location, and availability providing a full list of current car inventory.

In the User Database we store a user's email, drivers license, location (if enabled), and a key to whatever car is currently being rented by the user.

A possible alternative that we could have used in the technology in our design could have been using noSQL to manage our data, possibly using an approach like Redis. Additionally, some alternatives we could have used in the organization of data would be using separating the User database into 2 separate databases, one for employees and one for customers. Single database for both makes for simpler implementation with less overhead however does lead to sparse data with null values that don't apply to employees. Storing payment data directly could have also been a potential design choice within our payment database which would give our company more direct control. This however as a trade off could expose the company to potential payment compliance issues that are currently avoided by passing responsibility off to third party software specializing in payment security.

Development plan and timeline

Partitioning of tasks:

Rami: Documentation & Software Summary - assist in UML

Hussein: Primary UML

Aidan: Software Architectural Diagram - assist in UML

William: Primary UML

Emily: Class and Method Descriptions - assist in UML

Team member responsibilities:

Submission of final project: Emily

Verify Git Organization: Aidan

Assist with Git Organization: Hussein

Finalizing UML: William & Rami